

Layouts & Views (Router)

INFO

Lesinhoud

Er wordt gewerkt met Vue Router om verschillende layouts en views te maken binnen een Vue-applicatie.

Doelstellingen

De student(e) kan:

- Werken met een Command Line Interface (CLI) / Node Package Manager (NPM) voor het installeren, updaten en verwijderen van pakketten en modules binnen een project.
- Een single-page frontend applicatie ontwikkelen met Vue.js.
- Zelfstandig de bovenstaande technologieën combineren tot een werkende webapplicatie.

▼ Inhoud

- [Wat is Vue Router?](#)
- [Installeren van Vue Router](#)
- [Projectstructuur bekijken](#)
- [Router toevoegen aan Vue-applicatie](#)
- [RouterView-component](#)
 - [Eager Loading vs Lazy Loading](#)
- [Navigeren met RouterLink](#)
- [Layouts](#)

- Views
- Conclusie

Wat is Vue Router?

Vue Router is de officiële router voor Vue.js. Hiermee stel je jezelf in staat om verschillende views (pagina's) en layouts te creëren binnen een Vue-applicatie. Je beheert de navigatie tussen de verschillende views en toont de juiste componenten op basis van de URL.

Installeren van Vue Router

Vue Router kan je apart installeren via NPM. Binnen dit opleidingsonderdeel selecteer je Vue Router tijdens het aanmaken van een nieuw Vue-project met `npm create vue@latest`, waardoor installatie en configuratie automatisch plaatsvinden.

```
npm create vue@latest
```

Noem het project `router-voorbeeld`.

```
sh
Vue.js – The Progressive JavaScript Framework
◆ Project name (target directory):
  router-voorbeeld
```

Vink **Linter**, **Prettier** en **Vue Router** aan.

sh

◆ Select features to include in your project: (↑/↓ to navigate, space to select, a to toggle all, enter to confirm)

- | ☐ TypeScript
- | ☐ JSX Support
- | ☒ Router (SPA development)
- | ☐ Pinia (state management)
- | ☐ Vitest (unit testing)
- | ☐ End-to-End Testing
- | ☒ Linter (error prevention)
- | ☒ Prettier (code formatting)
- |

Klik bij de andere opties op Enter om de standaardwaarden te behouden.

sh

◆ Select experimental features to include in your project: (↑/↓ to navigate, space to select, a to toggle all, enter to confirm)

| none

◆ Skip all example code and start with a blank Vue project?

| No

Projectstructuur bekijken

Merk op dat er een map genaamd `router` is toegevoegd in de `src` - map.

js

```
import { createRouter, createWebHistory } from "vue-router";
```

```
import HomeView from "../views/HomeView.vue";

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: "/",
      name: "home",
      component: HomeView,
    },
    {
      path: "/about",
      name: "about",
      // route level code-splitting
      // this generates a separate chunk (About.
      [hash].js) for this route
      // which is lazy-loaded when the route is visited.
      component: () => import("../views/AboutView.vue"),
    },
  ],
});

export default router;
```

In dit bestand wordt een object genaamd `router` aangemaakt via de `createRouter()`-functie. Deze functie is beschikbaar in de `vue-router`-module (deze module is als dependency toegevoegd aan het `package.json`-bestand door de selectie die gemaakt is tijdens het aanmaken van het project).

Dit object wordt vervolgens geëxporteerd zodat het in `src/main.js` kan worden geïmporteerd.

De werking van de `routes`-array bekijk je later. Voorlopig kan je onthouden dat het `router`-object de verschillende routes (views) van de applicatie beheert.

Router toevoegen aan Vue-applicatie

In `src/main.js` wordt het `router` -object geïmporteerd en toegevoegd aan de Vue-applicatie via de `.use()` -methode.

```
import './assets/main.css'

import { createApp } from 'vue'
import App from './App.vue'
import router from './router'

const app = createApp(App)

app.use(router)

app.mount('#app')
```

js

Dit zorgt ervoor dat je router-functionaliteit in de Vue-applicatie beschikbaar hebt.

RouterView-component

De `RouterView` -component gebruik je als een placeholder voor de component die overeenkomt met de huidige route.

In `src/App.vue` wordt de `RouterView` -component gebruikt in het `template` -gedeelte.

In het `script` -gedeelte wordt de `RouterView` -component geïmporteerd uit de `vue-router` -module.

```
<script setup>
import { RouterLink, RouterView } from 'vue-router'
```

vue

```
import HelloWorld from './components/HelloWorld.vue'  
</script>
```

In het `template` -gedeelte voeg je de `RouterView` -component toe op de locatie waar je de component van de huidige route weergeeft.

```
<template>  
  <header>  
      
  
    <div class="wrapper">  
      <HelloWorld msg="You did it!" />  
  
      <nav>  
        <RouterLink to="/">Home</RouterLink>  
        <RouterLink to="/about">About</RouterLink>  
      </nav>  
    </div>  
  </header>  
  
  <RouterView />  
</template>
```

Wanneer je de applicatie uitvoert, geeft de `RouterView` -component de component weer die overeenkomt met de huidige route.

In de `routes` -array in `src/router/index.js` zijn twee routes gedefinieerd:

```

import { createRouter, createWebHistory } from 'vue-
router'
import HomeView from '../views/HomeView.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'home',
      component: HomeView,
    },
    {
      path: '/about',
      name: 'about',
      // route level code-splitting
      // this generates a separate chunk (About.
[hash].js) for this route
      // which is lazy-loaded when the route is visited.
      component: () => import('../views/AboutView.vue'),
    },
  ],
})

export default router

```

De route met het pad `/` toont de `HomeView` -component en de route met het pad `/about` toont de `AboutView` -component.

Je kan de applicatie testen door deze op te starten:

```
npm run dev
```

Navigeer naar de url die wordt getoond (meestal `http://localhost:5173`). Op de homepage (`/`) geeft de `RouterView` -

component de `HomeView` -component weer.

Bezoek vervolgens de `/about` -view `http://localhost:5173/about` .

De `AboutView` -component wordt vervolgens weergegeven op de locatie van de `RouterView` -component.

Oefening

Voeg een nieuwe view-component toe genaamd `TestView.vue` in de `src/views` -map. Deze component toont een `<h1>` -element met de tekst `Dit is de testpagina` .

Voeg een nieuwe route toe in `src/router/index.js` met het pad `/test` die de `TestView` -component toont. Gebruik de bestaande Home-route als voorbeeld.

Controleer het resultaat door naar `/test` te navigeren in de browser. De tekst `Dit is de testpagina` moet dan zichtbaar zijn.

Eager Loading vs Lazy Loading

In de `routes` -array worden twee verschillende manieren getoond waarop je componenten laadt.

Je importeert de `HomeView` -component direct bovenaan het bestand. Je kent deze component toe aan de route via de `component` -eigenschap. Dit heet `Eager Loading` , omdat je de component direct laadt wanneer de applicatie start, ongeacht of de gebruiker deze route bezoekt of niet.

Je importeert de `AboutView` -component niet direct. In plaats daarvan gebruik je een anonieme arrowfunctie die de component importeert wanneer je de route bezoekt. Dit heet `Lazy Loading` . Je laadt de component pas wanneer de gebruiker daadwerkelijk naar deze route navigeert. Dit kan de initiële laadtijd van de applicatie verbeteren, vooral als er veel routes en componenten zijn.

Voor kleine applicaties met weinig routes is **Eager Loading** vaak voldoende. Voor grotere applicaties met veel routes kan je **Lazy Loading** gebruiken om de prestaties te verbeteren door alleen de benodigde componenten te laden wanneer ze nodig zijn.

Oefening

Wijzig de **TestView** -route die eerder werd toegevoegd zodat deze gebruik maakt van **Lazy Loading** in plaats van **Eager Loading**.

Navigeren met RouterLink

In **src/App.vue** is een navigatiemenu aanwezig met twee **RouterLink** -componenten.

In het **script** -gedeelte wordt de **RouterLink** -component geïmporteerd uit de **vue-router** -module.

```
<script setup>
import { RouterLink, RouterView } from 'vue-router'
import HelloWorld from './components/HelloWorld.vue'
</script>
```

vue

In het **template** -gedeelte worden de **RouterLink** -componenten gebruikt om navigatielinks te maken. De **to** -eigenschap specificeert het pad waarnaar genavigeerd wordt wanneer op de link wordt geklikt.

```
<template>
  <header>
    

    <div class="wrapper">
        <HelloWorld msg="You did it!" />

        <nav>
            <RouterLink to="/">Home</RouterLink>
            <RouterLink to="/about">About</RouterLink>
        </nav>
    </div>
</header>

<RouterView />
</template>

```

Bij het klikken op een `RouterLink` wordt de URL bijgewerkt door Vue Router en wordt de overeenkomstige component in de `RouterView` weergegeven.

Oefening

Voeg een nieuwe `RouterLink` toe in het navigatiemenu die naar de `/test` -route navigeert. Bevestig dat het werkt door op de link te klikken en te controleren of de `TestView` -component wordt weergegeven.

Layouts

Momenteel heb je één layout voor de hele applicatie. Hetgeen je op alle views weergeeft, staat in `src/App.vue`. Hetgeen dat tussen de pagina's verschilt, wordt weergegeven op de locatie van de `RouterView` -component.

```
vue
<template>
  <header>
    

    <div class="wrapper">
      <HelloWorld msg="You did it!" />

      <nav>
        <RouterLink to="/">Home</RouterLink>
        <RouterLink to="/about">About</RouterLink>
      </nav>
    </div>
  </header>

  <RouterView />
</template>
```

Nu wordt getoond hoe je verschillende layouts maakt, bijvoorbeeld een layout voor gewone gebruikers en een layout voor beheerders (admin).

Maak een nieuwe map aan genaamd `layouts` in de `src`-map. Maak vervolgens twee nieuwe componenten aan: `UserLayout.vue` en `AdminLayout.vue`.

```
mkdir src/layouts
```

sh

```
touch src/layouts/UserLayout.vue
```

sh

```
touch src/layouts/AdminLayout.vue
```

sh

Verplaats de inhoud van `src/App.vue` naar
`src/layouts/UserLayout.vue`.

UserLayout.vue:

```
<script setup>
import { RouterLink, RouterView } from "vue-router";
import HelloWorld from "../components/HelloWorld.vue";
</script>

<template>
  <header>
    

    <div class="wrapper">
      <HelloWorld msg="You did it!" />

      <nav>
        <RouterLink to="/">Home</RouterLink>
        <RouterLink to="/about">About</RouterLink>
      </nav>
    </div>
  </header>

  <RouterView />
</template>

<style scoped>
```

vue

```
header {
  line-height: 1.5;
  max-height: 100vh;
}

.logo {
  display: block;
  margin: 0 auto 2rem;
}

nav {
  width: 100%;
  font-size: 12px;
  text-align: center;
  margin-top: 2rem;
}

nav a.router-link-exact-active {
  color: var(--color-text);
}

nav a.router-link-exact-active:hover {
  background-color: transparent;
}

nav a {
  display: inline-block;
  padding: 0 1rem;
  border-left: 1px solid var(--color-border);
}

nav a:first-of-type {
  border: 0;
}

@media (min-width: 1024px) {
  header {
    display: flex;
    place-items: center;
  }
}
```

```

padding-right: calc(var(--section-gap) / 2);
}

.logo {
margin: 0 2rem 0 0;
}

header .wrapper {
display: flex;
place-items: flex-start;
flex-wrap: wrap;
}

nav {
text-align: left;
margin-left: -1rem;
font-size: 1rem;

padding: 1rem 0;
margin-top: 1rem;
}
}
</style>

```

In `src/App.vue` blijft enkel de `RouterView` -component over.

```

<template>
  <RouterView />
</template>

```

vue

script/template/style

Enkel het `template` -gedeelte bevat inhoud. Lege `script` - en `style` - gedeeltes zijn dus overbodig en kunnen verwijderd worden.

De `UserLayout` -component gebruik je nu voor de routes die je eerder hebt gedefinieerd.

`src/router/index.js` :

```
import { createRouter, createWebHistory } from 'vue-router'
import UserLayout from '../layouts/UserLayout.vue'
import HomeView from '../views/HomeView.vue'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      component: UserLayout,
      children: [
        {
          path: '',
          name: 'home',
          component: HomeView,
        },
        {
          path: 'about',
          name: 'about',
          // route level code-splitting
          // this generates a separate chunk (About.[hash].js) for this route
          // which is lazy-loaded when the route is
          // visited.
          component: () =>
            import('../views/AboutView.vue'),
        },
      ],
    },
  ],
})
```

```
export default router
```

Merk op dat hetgeen je eerst aan `routes` toekende, nu aan de `children`-eigenschap van een route wordt toegekend die de `UserLayout`-component gebruikt.

Op `/` gebruik je de `UserLayout`-component, deze wordt ingeladen op de locatie van de `RouterView` in `src/App.vue`. Vervolgens match je het pad met de child-routes om te bepalen welke component op de locatie van de `RouterView` in `UserLayout.vue` verschijnt. Er zijn twee child-routes:

- De lege child-route (`path: ''`) die de `HomeView`-component toont.
- De `about` child-route (`path: 'about'`) die de `AboutView`-component toont.

Child Routes

Je kan child routes ook nesten binnen andere child routes, waardoor er meerdere niveaus van layouts mogelijk zijn.

Binnen dit opleidingsonderdeel gebruik je voornamelijk enkel één niveau van layouts.

Oefening - extra layout

- Voeg een component toe genaamd `AdminLayout.vue` in de map `src/layouts`.
- Voeg twee componenten toe aan de map `src/views`: `AdminDashboard.vue` en `AdminSettings.vue`. Beide componenten tonen een `<h1>`-element met respectievelijk de teksten `Admin Dashboard` en `Admin Settings`.
- Voeg in `src/router/index.js` een nieuwe route toe met het pad `/admin` die de `AdminLayout`-component gebruikt. Definieer twee child-routes: één voor het dashboard (`/admin/dashboard`) en één voor de

instellingen (`/admin/settings`), die respectievelijk de `AdminDashboard` - en `AdminSettings` -componenten tonen.

- Voeg in `AdminLayout.vue` een navigatiemenu toe met `RouterLink` - componenten die naar de `/admin/dashboard` - en `/admin/settings` - routes navigeren.
- Voeg in `AdminLayout.vue` ook een `RouterView` -component toe om de child-routes weer te geven.
- Voeg in `AdminLayout.vue` een footer toe met de tekst `Admin Panel Footer` . Plaats deze onder de `RouterView` -component.

Test het resultaat door naar `/admin/dashboard` en `/admin/settings` te navigeren in de browser. Hier zou de navigatiebalk van de adminlayout, de Dashboard- of Settings-inhoud en de footer zichtbaar moeten zijn.

Je kan nu verschillende layouts toepassen op verschillende delen van de applicatie, bijvoorbeeld een layout voor een gewone gebruiker en een layout voor een beheerder.

Wanneer je op elke pagina iets weergeeft, zoals een globale banner bovenaan met een belangrijke mededeling, kan je dit in

`UserLayout.vue` en `AdminLayout.vue` plaatsen. Om duplicatie te vermijden, kan je de banner ook in `App.vue` plaatsen zodat deze op elke pagina verschijnt, ongeacht de gebruikte layout.

```
<template>
  <RouterView />
  <div class="banner">
    <p>Welkom bij de Vue Router Applicatie</p>
  </div>
</template>

<style scoped>
.banner {
  position: fixed;
  top: 0;
```

vue

```

    left: 0;
    background-color: #42b983;
    color: white;
    text-align: center;
    padding: 1rem;
    font-size: 1.2rem;
    width: 100vw;
  }
</style>

```

Wanneer je de verticale ruimte verkleint, kan de banner overlappen met de inhoud van de pagina. Om dit op te lossen voeg je globale CSS toe om de inhoud iets naar beneden te duwen.

```

<template>
  <RouterView />
  <div class="banner">
    <p>Welkom bij de Vue Router Applicatie</p>
  </div>
</template>

<style scoped>
.banner {
  position: fixed;
  top: 0;
  left: 0;
  background-color: #42b983;
  color: white;
  text-align: center;
  padding: 1rem;
  font-size: 1.2rem;
  width: 100vw;
}
</style>

<style>
body {

```

```
padding-top: 5rem; /* hoogte van de banner + wat extra  
witruimte */  
}  
</style>
```

scoped/global styles

Wanneer je stijlen in een `.vue` -bestand toevoegt, gebruik je normaal `<style scoped>`. Hiermee zorg je ervoor dat de stijlen enkel van toepassing zijn op de component zelf.

Omdat je de `body` -tag wilt stylen, voeg je een `style` -tag zonder `scoped` -attribuut toe. Hierdoor pas je de stijlen globaal toe op de hele applicatie en niet enkel op de component zelf.

Oefening - globale styling

De code hierboven werkt, maar het is niet ideaal om globale styling in een component te plaatsen. Verplaats de globale styling naar een locatie die beter geschikt is voor globale stijlen.

▼ Oplossing

Je kan de globale styling verplaatsen naar `src/assets/main.css`, aangezien dit bestand al wordt geïmporteerd in `src/main.js` en bedoeld is voor globale stijlen.

Views

Er zijn al enkele views aangemaakt in de `src/views` -map:

- `HomeView.vue`
- `AboutView.vue`
- `AdminDashboard.vue` (indien toegevoegd tijdens de oefening)

- `AdminSettings.vue` (indien toegevoegd tijdens de oefening)

Views zijn, net als layouts, gewoon Vue-componenten. Het verschil is dat je een layout gebruikt om de algemene structuur van een pagina te definiëren (zoals header, footer, navigatie), terwijl je een view gebruikt voor de specifieke inhoud van een pagina.

Je koppelt routes in de router-configuratie aan views. Binnen de gemaakte setup koppel je Layout-componenten aan routes, terwijl je views aan child-routes binnen die layout-route koppelt.

Conclusie

De Vue Router stelt je in staat om binnen één applicatie meerdere layouts en views te gebruiken. Dit maakt het mogelijk om een gestructureerde en georganiseerde applicatie te bouwen waarbij verschillende delen van de applicatie verschillende layouts kunnen hebben, terwijl je de specifieke inhoud per pagina via views beheert.

Een view is een Vue-component die je gebruikt om de inhoud van een specifieke pagina weer te geven. Dit kan HTML zijn, maar ook andere componenten die samen de inhoud van die pagina vormen.

Previous page
[Oefeningen](#)

Next page
[Oefeningen](#)